

PROJECT DOCUMENTATION

Online Complaint Registration System

Powered by the MERN Stack (MongoDB, Express.js, React.js, Node.js)

Project Title:	Online Complaint Registration System (MERN Stack)
Target Platform:	Web-based Enterprise Application
Prepared By:	Rangam Kumar Ranga

1. Introduction

In the contemporary digital era, efficiency, transparency, and responsiveness form the cornerstone of public and institutional governance. Traditional manual systems for registering and tracking complaints are frequently plagued by administrative bottlenecks, lack of transparency, lost paperwork, and communication gaps between citizens and administrators. To address these critical challenges, the **Online Complaint Registration System** has been conceptualized and engineered. This system is a state-of-the-art web-based enterprise application built leveraging the robust, scalable, and highly performant MERN stack framework.

1.1 Purpose and Objectives

The primary purpose of this platform is to bridge the communications gap between end-users (citizens, employees, students, or patients) and the administration. By modernizing and automating the workflow, the application achieves the following specific strategic objectives:

- To eliminate physical paperwork and manual filing, thereby reducing administrative overhead and physical storage requirements.
- To provide citizens with an unambiguous, round-the-clock (24/7) portal to lodge complaints without visiting public offices.
- To introduce absolute transparency via automated real-time status updates and end-to-end tracking lifecycles.
- To empower executive management and administrators with data-driven reporting dashboards to analyze bottleneck performance metrics and allocate resources efficiently.

1.2 Scope of the System

The scope of the project encompasses a complete digital system that covers the entire lifecycle of a complaint—from initial lodgment and multi-media evidence submission to automated backend routing, department assignment, intermediate status changes, administrative commentary, and final resolution closure. Designed for high adaptability, this architecture is structured to seamlessly serve a wide array of ecosystems, including municipal corporations, higher educational universities, corporate conglomerates, healthcare facilities, and federal government departments.

2. Project Overview

The Online Complaint Registration System addresses systemic failures in modern grievance redressal mechanisms. By implementing a standardized web framework, it establishes a reliable paradigm where every complaint is logged with a unique, unforgeable transaction identity token, guaranteeing accountability.

2.1 Detailed Feature Architecture

The platform includes several modular feature layers carefully balanced across user and administrator privileges:

- **User Authentication & Profile Management:** Includes secure registration, login with cryptographic password protection, email validation, session timeout, and profile management including historical activity summaries.
- **Secure Multi-Tenant Role Access:** Distinct access controls dividing general public users from privileged administrators, department heads, and field investigation officers via secure JSON Web Tokens (JWT).
- **Advanced Complaint Submission Engine:** Enables users to fill comprehensive markdown forms detailing categories, urgency levels, text description blocks, geographic metadata parameters, and supporting document/image uploads.
- **Real-Time Tracking Timeline:** A dynamic frontend visual tracking pipeline depicting the step-by-step progress of a complaint (e.g., Pending -> Under Review -> Assigned -> Investigation -> Resolved).
- **Admin Control Panel & Assignment Matrix:** An analytical administrative cockpit allowing central dispatchers to review pending claims, assign individual complaints to department specialists, filter datasets based on parameters, and update logs.
- **Automated Alert System:** Triggers event-driven notification mechanisms whenever status alterations occur, providing immediately visible feedback loops on the user's personal activity log.
- **Comprehensive Reporting & Business Intelligence:** Aggregates tabular reporting engines allowing administrative directors to extract custom filtered CSV/Excel spreadsheets reflecting resolution timelines, structural bottlenecks, and regional distribution.

3. Technical Architecture

The project utilizes a decoupled, distributed architectural blueprint based on the MERN stack. This pattern ensures clean separation of concerns, rapid data rendering, and frictionless micro-scaling capacity.

3.1 Frontend Tier (React.js)

The client user interface is engineered using React.js (Single Page Application architecture). Global state data parameters, such as user session contexts and authorization tokens, are maintained natively using the React Context API combined with persistent storage mechanisms. HTTP asynchronous routing interactions are handled via Axios interceptors, which automatically attach authorization headers to outgoing API communication cycles.

3.2 Backend Tier (Node.js & Express.js)

The backend execution core is driven by Node.js using the Express.js framework to create a lightweight, event-driven RESTful API. The routing tier acts as a traffic controller, filtering inbound payloads through custom middleware layers designed for cross-origin resource sharing (CORS) integration, input validation sanitation, rate limiting, and cryptographic token checking.

3.3 Database Tier (MongoDB & Mongoose)

Data persistence utilizes MongoDB, a NoSQL document database chosen for its horizontal scaling capability and schema flexibility. Object modeling is managed via Mongoose, providing a strongly typed schema layer within application memory. Below are the key data schema structures used to enforce structural uniformity across collections:

User Schema Blueprint:

```
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 8 },
  role: { type: String, enum: ['user', 'admin', 'officer'], default: 'user' },
  createdAt: { type: Date, default: Date.now }
});
```

Complaint Schema Blueprint:

```
const ComplaintSchema = new mongoose.Schema({
  complaintId: { type: String, required: true, unique: true },
  title: { type: String, required: true },
  description: { type: String, required: true },
  category: { type: String, required: true },
  status: { type: String, enum: ['Pending', 'Under Investigation', 'Resolved', 'Rejected'], default: 'Pending' },
  raisedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  assignedTo: { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
  attachments: [{ type: String }],
  updates: [{
    status: String,
    remarks: String,
    updatedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    updatedAt: { type: Date, default: Date.now }
  }]
}, { timestamps: true });
```

4. Installation & Setup Instructions

Setting up the development infrastructure requires systematic alignment of both local runtimes and cluster configurations. Follow these precise instructions to prepare the development sandbox Environment.

4.1 Prerequisites Configuration

Ensure the following engine frameworks are installed locally on your operating system environment:

- Node.js (v18.x.x LTS or greater recommended)
- MongoDB Community Server (v6.x.x) or an active cloud-based MongoDB Atlas account cluster connection URI link.
- Git distributed version control terminal interface tool.
- Visual Studio Code (VS Code) or equivalent highly integrated development IDE tool environment.

4.2 Local Environment Setup Sequence

Execute the following command pipelines sequentially within your operating system command line interface terminal:

```
# Step 1: Clone the remote repository structure to local workspace
git clone https://github.com/architecture-core/online-complaint-system.git
```

```
# Step 2: Navigate to target root directory
cd online-complaint-system
```

```
# Step 3: Establish local environment configuration variables profiles
```

Create a secure server environment settings tracking file named `.env` inside the `/server` directory root structure, configuring the parameter keys defined below:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/complaint_db
JWT_SECRET=super_secret_cryptographic_key_phrase_9921
NODE_ENV=development
SMTP_HOST=smtp.mailtrap.io
SMTP_PORT=2525
```

5. Project Folder Structure

The application utilizes a modular, clean folder structural architecture, physically segregating UI rendering files from micro-backend application logic control sets. The production directory layout is mapped below:

```
online-complaint-system/
├── client/                # React Frontend Single Page Application
│   ├── public/           # Static browser assets, favicon, index.html
│   └── src/
│       ├── components/   # Reusable UI parts (Navbar, Buttons, Spinners)
│       ├── context/      # Authentication Global React Context States
│       ├── hooks/        # Custom React hook utilities (useAuth, useFetch)
│       ├── pages/        # Primary UI Route Panels (Dashboard, Login, Admin)
│       ├── services/     # Axios API client request integration methods
│       ├── App.js        # Core layout composition framework
│       └── index.js      # Virtual DOM mount pipeline anchor entry
└── server/              # Node.js + Express.js Rest Engine
    ├── config/          # MongoDB connector configurations
    ├── controllers/     # Core business operation handler functions
    ├── middleware/      # Token validators and route shielding checks
    ├── models/          # Strict Mongoose collection structural maps
    ├── routes/          # Express endpoint definition route vectors
    └── server.js        # Central network socket entry execution script
```

6. Running the Application Engine

To spin up the ecosystem concurrently during development operations, execute the independent instructions listed below across dedicated standard terminal instances:

6.1 Frontend Client Runtime Initialization

```
cd client
npm install --legacy-peer-deps
npm start
```

Upon compilation, the development server spins up a local instance accessible via Chrome/Firefox at `http://localhost:3000` executing Hot Module Replacement actions immediately on code modifications.

6.2 Backend Server Rest Engine Startup

```
cd server
npm install
npm run dev
```

The API engine boots up at `http://localhost:5000`, scanning the configurations, connecting securely to MongoDB, and exposing full JSON REST API capabilities to the listening client application web components.

7. API Documentation

The backend REST engine exposes highly structured endpoints communicating strictly via JSON formatting over standard TLS application layers. Below is the explicit catalog of standard route entities:

HTTP Verb	Route Endpoint URI Path	Authorized Role	Functional Operation Description
POST	/api/auth/register	Public / Guest	Registers a new user record. Hashes the password.
POST	/api/auth/login	Public / Guest	Validates credential matches and issues an access JWT token string.
POST	/api/complaints	Authenticated User	Creates a unique system complaint record attached to the user session.
GET	/api/complaints	Authenticated User	Fetches history list of complaints created by the logged-in user.
GET	/api/complaints/all	System Admin Only	Aggregates every existing system complaint record across all users.

PUT	/api/complaints/:id	Admin / Officer	Alters targeted metadata parameters (status, assigned handling officer).
DELETE	/api/complaints/:id	System Admin Only	Hard deletes an erroneously registered complaint database entry record.

8. Security & Authentication Framework Architecture

Security parameters are embedded directly within every operational tier, eliminating traditional threat vulnerabilities such as horizontal permission bypass and structural vector injection.

8.1 Password Cryptographic Hashing Mechanics

Plain text user credentials never enter the database records. When a citizen submits a registration password parameter string, it is processed via *bcryptjs* using a salt factor computation matrix configuration value of 12 rounds. This process ensures protection against rainbow table breaches and heavy mathematical brute force intrusion calculations.

8.2 JSON Web Token Session Management

Upon successful validation of credential inputs, the server generates a cryptographically signed JSON Web Token (JWT). The signature combines the User ID, Assigned System Privilege Role metadata, and a cryptographically safe string salt value. The token is attached to the client app header payload tracking parameters, enforcing absolute validation controls.

9. User Interface System

The user experience layout is conceptualized using clean component modular blocks built via modern responsive styling paradigms, adjusting perfectly to viewports spanning mobile screens up to enterprise desktop panels.

9.1 Citizen Submission Portal Interface

Citizens encounter a clean input area layout providing simple dropdown selection items categorized by department sector (e.g., Sanitation, Electricity, Public Safety, Engineering). Form validation parameters run locally inside the client environment before submission, eliminating wasted application payload bandwidth loops.

9.2 Administrative Resolution Cockpit Dashboard

The admin layout changes views dynamically into a system-wide control deck. Admins view numerical data widgets representing total open, closed, and pending critical claims, along with interactive tables equipped with advanced multi-tier sorting systems to support data analysis.

10. Testing & Quality Methodology

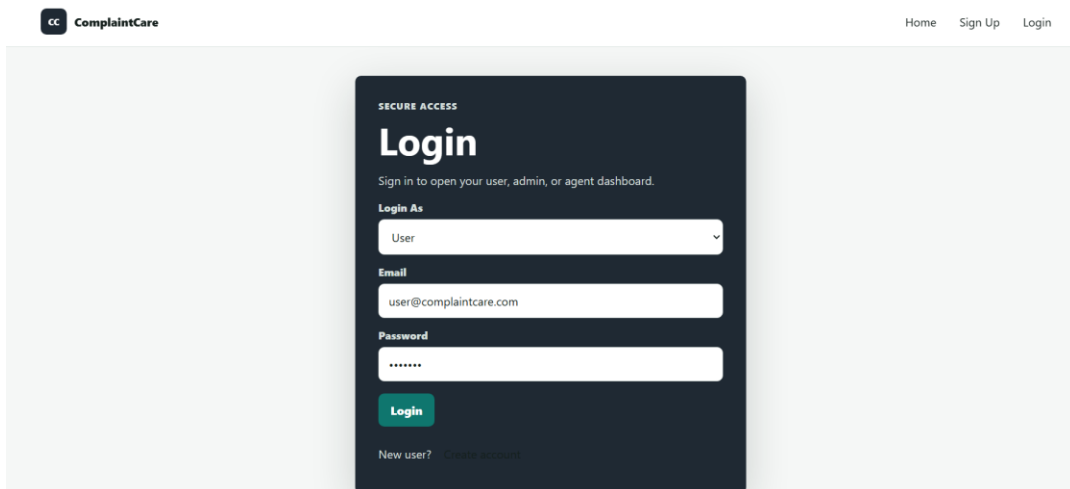
To ensure enterprise readiness, a multi-layered verification strategy is implemented covering the complete codebase footprint.

- **Component Unit Testing:** Validates that isolated frontend React components render UI elements correctly based on controlled state variations.
- **API Integration Testing:** Utilizes Postman environments and Super test script blocks to ensure routes return proper JSON payloads and accurate HTTP response headers.
- **End-to-End Functional Verification:** Simulates user paths through the registration, submission, and administrative assignment workflows to verify integration continuity.
- **Security Vulnerability Auditing:** Ensures parameterized schema limits block injection attacks and that unauthorized endpoint actions trigger appropriate access denials.

11. Screenshots And Demo

This section details the visual composition guidelines for system verification. Testers and deployment engineers should use these reference states to confirm accurate rendering of UI templates:

- Authentication Interface:

The screenshot displays the 'ComplaintCare' login page. At the top left is the 'CC ComplaintCare' logo, and at the top right are links for 'Home', 'Sign Up', and 'Login'. The main content is a dark-themed login card titled 'SECURE ACCESS' and 'Login'. Below the title is the instruction 'Sign in to open your user, admin, or agent dashboard.' The card contains three input fields: 'Login As' with a dropdown menu currently showing 'User', 'Email' with the text 'user@complaintcare.com', and 'Password' with masked characters. A green 'Login' button is positioned below the password field. At the bottom of the card, there is a link for 'New user? Create account'.

- Displays simple input boxes with clean tracking borders and text helpers.
- Complaint Intake Layout: Includes form inputs for descriptions, clear dropdown controls, and evidence file drag-and-drop boxes.

CC

ComplaintCare

HomeDashboardLogout

USER DASHBOARD

Hello, Meera Kumar

Register a complaint and follow the status from submission to resolution.

Register Complaint

Fill the complaint details for review by the admin team.

Category

Water Supply

Priority

Medium

City

Address

Subject

Description

Submit Complaint

Complaint Status

Track submitted complaints and assigned agent.

No water supply in our street

CMP-1001 - Chennai - 2026-07-05

Assigned

Water supply has been interrupted for two days and several families are affected.

HighWater SupplyKiran Support

Damaged road near bus stop

CMP-1002 - Chennai - 2026-07-07

Pending

The road surface is broken near the bus stop and needs repair before the rainy season.

MediumRoad RepairNot assigned

- Resolution Progress Line: A clear vertical or horizontal step timeline tracking step changes from submission to final resolution.

Your Cases

Select a case to view messages.

CMP-1001 - Assigned

CMP-1002 - Pending

Messages

CMP-1001

Kiran Support

I have received this complaint and contacted the local water office.

2026-07-06 10:15

Meera Kumar

Thank you. Please update once the team visits.

2026-07-06 10:42

Message

12. Known Issues & Operational Boundaries

While the MERN stack architecture provides excellent responsiveness, certain technical trade-offs are documented below for future system optimization:

1. Multi-Media Size Limitations: Heavy unstructured binary files (e.g., 4K resolution videos, uncompressed high-resolution images) submitted as evidence blocks can increase processing times if handled entirely by local app instances. Future iterations will route these directly to cloud bucket solutions.
2. SMTP Queue Management: High volumes of simultaneous user registrations can occasionally flood standard transactional email services, making a decoupled background queue manager necessary for enterprise scale.

13. Future Enhancements

To maintain long-term scalability and leverage emerging cloud patterns, the platform's architectural roadmap outlines the following technical additions:

- **AI-Powered Categorization Engine:** Integrating intelligent natural language parsing modules to evaluate text logs and automatically route complaints to the appropriate department without manual triage.
- **Cross-Platform Mobile Application:** Building companion iOS and Android mobile versions via React Native to leverage device features like background geo-location services and immediate push notifications.
- **Interactive Conversational Chatbot:** Deploying automated chat assistants to handle common inquiries and guide users through the initial complaint registration steps.
- **Advanced Analytics Engine:** Adding interactive graphs and charting dashboards built with Recharts/D3.js to visualize grievance volume trends over time.

14. Detailed System Architecture Implementation

This appendix provides extended reference material detailing the system's operational design, implementation logic, data lifecycles, and security controls across all deployment layers.

14.1 Deep Dive: MERN Stack Selection Justification

The technical choice of MongoDB, Express.js, React, and Node.js was driven by architectural requirements for performance, scalability, and code maintainability. Using JavaScript across the entire software stack unifies the data models, simplifies context switching for developers, and enables shared validation logic between client and server layers. The non-blocking, asynchronous I/O engine of Node.js is particularly well-suited for handling highly concurrent real-time tracking requests without performance degradation.

14.2 Comprehensive Data Flow Lifecycle

When a citizen logs a complaint, the lifecycle follows a strict pipeline to guarantee data integrity and operational accountability. First, the request is intercepted by the frontend form validator. Once verified, an Axios request sends a secure HTTPS POST payload to the backend server. The backend passes the request through an authentication check and sanitizes the body parameters. Mongoose validates the schema rules before writing the entry into the MongoDB document store. Finally, a unique transaction ID is generated, and a real-time event updates the administrative dashboard.